Query Processing
External Sort–Merge Algorithm

---

1. Introduction

When a relation is too large to fit entirely in main memory, we cannot use standard in-memory sorting algorithms like QuickSort. Instead, we must use **external sorting** techniques that minimize disk I/O.

The most efficient method is the **External Sort–Merge Algorithm**, which works in two phases:

1. **Run Generation:** Create sorted chunks that fit in memory
2. **Merge Phase:** Combine sorted chunks into a single sorted output

Key Terminology

- M: Number of memory buffer blocks available (e.g., $M = 3$ means 3 blocks fit in RAM)
- b_r: Total number of blocks in the relation to be sorted
- b_b: Number of blocks read/written per I/O operation (buffer size)
- **Run**: A sorted portion of the data that fits in memory
- **N-way Merge**: Merging N sorted runs simultaneously

2. Phase 1: Run Generation

Goal: Divide the relation into sorted chunks called **runs**.

Algorithm:

1. Read M blocks from disk into memory
2. Sort these blocks in memory (using QuickSort, etc.)
3. Write the sorted blocks to disk as run R_i
4. Repeat until all data is processed

Number of Runs Created

$$\text{Number of Runs} = \left\lceil \frac{b_r}{M} \right\rceil$$

Example: If $b_r = 12$ blocks and $M = 3$, we create $\lceil 12/3 \rceil = 4$ runs.

Concrete Example: Run Generation

Scenario: Sort a relation with 12 blocks, memory can hold 3 blocks ($M = 3$).
Unsorted Data (12 blocks):

17	3	29	8	41	12	5	33	19	27	2	44

Step-by-step:

1. Read blocks 1-3: [17, 3, 29] → Sort → Write R_0: [3, 17, 29]
2. Read blocks 4-6: [8, 41, 12] → Sort → Write R_1: [8, 12, 41]
3. Read blocks 7-9: [5, 33, 19] → Sort → Write R_2: [5, 19, 33]
4. Read blocks 10-12: [27, 2, 44] → Sort → Write R_3: [2, 27, 44]

Result: 4 sorted runs (R_0, R_1, R_2, R_3), each containing 3 sorted blocks.

3. Phase 2: Merge Phase

Goal: Combine all runs into a single sorted output.

3.1 Case 1: All Runs Fit ($N < M$)

If the number of runs N is less than M, we can merge all runs in **one pass**:

1. Allocate one input buffer for each of the N runs
2. Allocate one output buffer for the merged result
3. Repeatedly select the smallest element across all buffers and write to output

Concrete Example: Single-Pass Merge

Continuing our example: 4 runs, $M = 5$ (so $N < M$).
Initial State:

R_0	R_1	R_2	R_3
3	**8**	**5**	**2**
17	12	19	27
29	41	33	44

Merge Process:

1. Compare front elements: [3, 8, 5, 2] → Min = **2** from R_3 → Output: [2]
2. R_3 advances to 27. Compare: [3, 8, 5, 27] → Min = **3** → Output: [2, 3]
3. R_0 advances to 17. Compare: [17, 8, 5, 27] → Min = **5** → Output: [2, 3, 5]
4. Continue until all runs exhausted...

Final Output: [2, 3, 5, 8, 12, 17, 19, 27, 29, 33, 41, 44]

3.2 Case 2: Too Many Runs ($N \geq M$)

If $N \geq M$, we cannot merge all runs at once. We need **multiple merge passes**.

Key Constraint:

$$(M-1) \text{ input buffers} + 1 \text{ output buffer} = M \text{ total buffers}$$

So we can merge at most $(M-1)$ runs per pass.

Concrete Example: Multi-Pass Merge

Scenario: 8 runs, $M = 3$ (can only merge 2 runs at a time).
Pass 1: Merge pairs of runs

- $R_0 + R_1 \to R'_0$
- $R_2 + R_3 \to R'_1$
- $R_4 + R_5 \to R'_2$
- $R_6 + R_7 \to R'_3$

Result: 4 runs
Pass 2: Merge pairs again

- $R'_0 + R'_1 \to R''_0$
- $R'_2 + R'_3 \to R''_1$

Result: 2 runs
Pass 3: Final merge

- $R''_0 + R''_1 \to$ Sorted Output

4. Cost Analysis

4.1 Number of Merge Passes

Each pass reduces the number of runs by a factor of $(M-1)$:

$$\boxed{P = \left\lceil \log_{M-1}\left(\frac{b_r}{M}\right) \right\rceil}$$

Example: $b_r = 1000$, $M = 11$

$$P = \left\lceil \log_{10}\left(\frac{1000}{11}\right) \right\rceil = \lceil \log_{10}(91) \rceil = \lceil 1.96 \rceil = 2 \text{ passes}$$

4.2 Total Block Transfers

Block Transfer Formula

$$\text{Total Transfers} = b_r \times (2\lceil \log_{M-1}(b_r/M) \rceil + 1)$$

Breakdown:

- Run generation: Read all blocks + Write all blocks = $2b_r$
- Each merge pass: Read all + Write all = $2b_r$ per pass
- Final pass output can be pipelined (not written to disk) = save b_r

4.3 Total Disk Seeks

Seek Formula

$$\text{Total Seeks} = 2\left\lceil \frac{b_r}{M} \right\rceil + \left\lceil \frac{b_r}{b_b} \right\rceil (2P - 1)$$

Breakdown:

- Run generation: 1 seek per run read + 1 seek per run write
- Merge passes: Seeks depend on buffer size b_b (larger = fewer seeks)

5. Complete Worked Example

Full Calculation

Given:

- Relation size: $b_r = 1000$ blocks
- Memory: $M = 5$ blocks
- Buffer size: $b_b = 1$ block

Step 1: Calculate Number of Runs

$$\text{Runs} = \left\lceil \frac{1000}{5} \right\rceil = 200 \text{ runs}$$

Step 2: Calculate Number of Merge Passes

$$P = \lceil \log_4(200) \rceil = \lceil 3.82 \rceil = 4 \text{ passes}$$

Step 3: Calculate Total Block Transfers

$$\text{Transfers} = 1000 \times (2 \times 4 + 1) = 1000 \times 9 = \mathbf{9000} \text{ transfers}$$

Step 4: Calculate Total Seeks

$$\text{Seeks} = 2 \times 200 + 1000 \times (2 \times 4 - 1) = 400 + 7000 = \mathbf{7400} \text{ seeks}$$

6. Summary

Key Takeaways

- External Sort-Merge handles relations larger than memory
- **Phase 1:** Create $\lceil b_r/M \rceil$ sorted runs
- **Phase 2:** Merge $(M-1)$ runs per pass until one remains
- More memory (M) $\rightarrow$ fewer passes $\rightarrow$ less I/O
- Larger buffer (b_b) $\rightarrow$ fewer seeks
- Cost: $O(b_r \cdot \log_{M-1}(b_r/M))$ block transfers